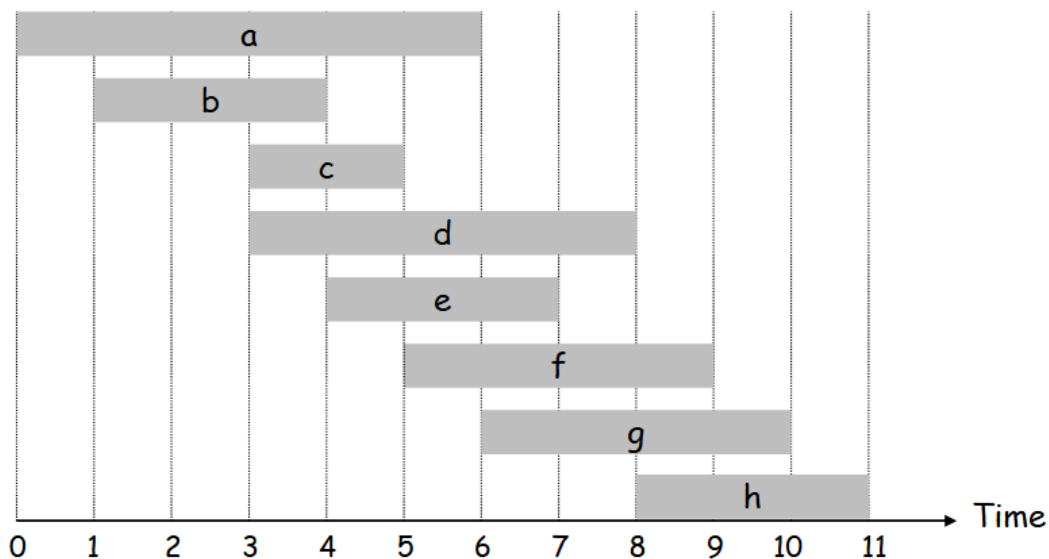


Greed

[04greed.pdf](#)

这一章的重点是贪心算法，贪心算法的思想无非就是：只顾眼前，不顾将来我们可以看一个例子：

区间调度问题



每个任务 j 有两个时间点：

- 开始时间： s_j
- 结束时间： f_j

所以每个任务都可以看成时间轴上的一个区间 $[s_j, f_j]$ 。如果两个任务，他们在时间上不重叠，那我们就可以说这两个任务是相容的。

本体的目标是，选出尽可能多的任务，使这些任务两两不重叠。（个数最多！）对于这样的要求，我们就面临着这样的问题：

- 任务 a 很长，虽然开始早，但它会挡住很多别的任务
- 有些短任务虽然看起来不起眼，但拼起来能选更多个

贪心模板：先决定一个任务的考察顺序，然后按这个顺序一个个看过去；如果当前任务和已经选的任务不冲突，就选它。

于是，贪心算法的关键就变成了，我们按照什么顺序来看任务呢？

- earliest start time 错

- shortest interval 错
- fewest conflicts 错

(以上反例均以在PPT中说明)

正确的贪心解法：

把所有任务**按结束时间从小到大排序**。然后从前往后扫描：

- 当前任务如果和已选任务兼容，就选它
- 否则跳过它

一个任务结束得越早，它对后面时间的占用越少，所以它给未来留下的空间越大。

- 先把任务按结束时间排序，使得

$$f_1 \leq f_2 \leq \cdots \leq f_n$$

- 初始化答案集合

$$A \leftarrow \emptyset$$

- 从第 1 个任务到第 n 个任务依次检查：

- 如果任务 j 和当前集合 A 兼容
- 就把 j 加入 A

- 最后返回 A

怎么样才算兼容呢？：

因为我们选出来的**任务本身就是按结束时间递增**、互不冲突的。（这意味着最后一个被选中的任务，结束时间一定是最晚的）所以只需要记住：

- **最后一个被选中的任务 j^***

然后判断当前任务 j 是否满足：

$$s_j \geq f_{j^*}$$

如果满足，就说明**它和前面所有已选任务都兼容**，可以直接选。

时间复杂度说明：

根据结束时间来排序，我们可以选择一些高级排序算法，时间复杂度为 $O(n \log n)$ ，排完序后我们做的操作就是从前向后扫描一遍，每次额外做判断兼容性，判断操作的复杂度为 $O(1)$ ，所以复杂度就是：

$$O(n \log n) + O(n) = O(n \log n)$$

算法正确性、最优性说明：

可行性：

显而易见，毕竟我们每加入新的任务的前提都是一定不会和已有的任务冲突，所以通过贪心算法构造出的任务集合，一定是可行的。

最优性：

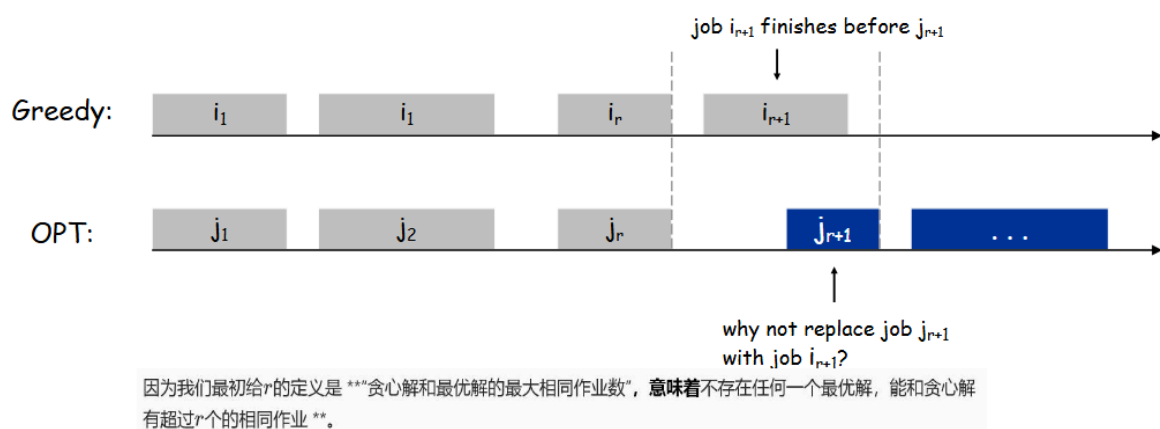
使用反证法来证明：取一个最优解，使得

$$i_1 = j_1, i_2 = j_2, \dots, i_r = j_r$$

对某个最大的 r 成立。这句话非常重要，它的意思是：在所有最优解里，我们挑一个和贪心解“前面重合得最多”的最优解。

也就是说：

- 前 r 个任务，贪心解和这个最优解完全一样
- 但是第 $r + 1$ 个任务开始，它们不同了（这是重点）
- 并且这个 r 已经尽可能大了



贪心总是选结束最早的那个而 j_{r+1} 也是一个可行候选所以贪心选出来的 i_{r+1} 的结束时间一定不晚于 j_{r+1} ，数学语言就是 $\leq$ ，所以第 $r + 1$ 个任务 j_{r+1} 可以换成 i_{r+1} 。

与前面 r 个任务仍保持兼容是显而易见的，现在我们的重点就是，任务换了之后后面的任务还都兼容吗？思考一下，其实也都兼容→OPT 原来第 $r + 1$ 个任务是 j_{r+1} ，后面的 $j_{r+2}, j_{r+3}, \dots$ 都能排在它后面，说明它们的开始时间都满足：

$$s(j_{r+2}) \geq f(j_{r+1}), \quad \text{等等}$$

而我们又知道：

$$f(i_{r+1}) \leq f(j_{r+1})$$

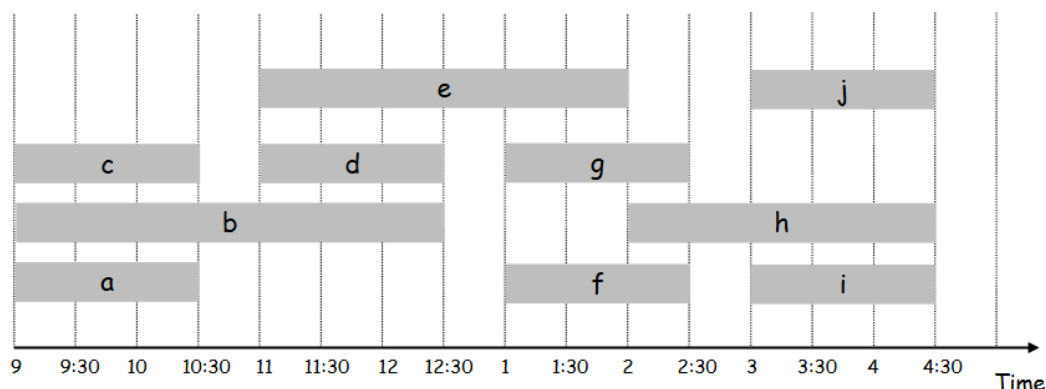
所以如果某个后续任务原来能接在 j_{r+1} 后面，那么它也一定能接在更早结束的 i_{r+1} 后面。于是我们发现OPT和贪心解相同的长度又加了一，然而我们的假设就是已经选了前缀长度最长的最优解了，矛盾！！

因此，贪心算法一定是最优的！

! Important

一句话总结任务安排的贪心算法：按照结束时间排序，从结束最早的开始安排，每次加入新的任务必须保证兼容，不兼容则按序向后找。

讲座安排问题：



在这个问题中，我们的任务是选出最小的教室数目，让这些讲座不冲突。首先有一点可以很确定，我们可以通过查看所有时段内，讲座重合的最大数，找到教室安排数目的下限，为什么呢？

我们把这个讲座重合的最大数称为 $depth$ 。

Note

因为重合最大数目代表讲座冲突的最坏情况，以这个题为例，最多有三个讲座同时开展，为了不让他们撞，**至少要用到三个教室**。

算法思想阐述：

```
Sort intervals by starting time so that  $s_1 \leq s_2 \leq \dots \leq s_n$ .  
 $d \leftarrow 0$   $\leftarrow$  number of allocated classrooms  
  
for  $j = 1$  to  $n$  {  
    if (lecture  $j$  is compatible with some classroom  $k$ )  
        schedule lecture  $j$  in classroom  $k$   
    else  
        allocate a new classroom  $d + 1$   
        schedule lecture  $j$  in classroom  $d + 1$   
         $d \leftarrow d + 1$   
}
```

首选按照讲座的**开始时间，从早到晚排序**。对于当前课程，把它放进任意一个与它兼容的教室；如果没有兼容教室，就新开一个教室。

对当前课程 j ，有两种情况：

情况 1：存在某个教室兼容也就是**这个教室上一门课已经结束了**。如果某个教室里最后一门课的结束时间 $\leq s_j$ ，那么当前课程 j 可以接着放进去。

于是：

- 不开新教室
- 直接复用这个教室

情况 2：没有任何教室兼容。说明所有已开教室中，最后一门课都还没结束。那当前课程没地方放，只能：

- 新开一个教室
- 把当前课程放进去

思路一览：为什么要按开始时间排序呢？

Note

这个要从直觉上理解。我们是按课程开始时间从早到晚一个个来安排。当处理到当前课程时：

- 比它更早开始的课已经都安排好了

- 比它更晚开始的课还没来，不需要考虑

所以当前真正的问题就是：这门课现在有没有空教室可用？如果有，就复用。如果没有，就只能新开。

算法时间复杂度分析：

$O(n \log n)$ ，对每个教室，维护最后一个区间的结束时间，把这些教室放进优先队列。

对于每一个已开教室，我们根本不关心它里面完整安排了哪些课。我们只关心一件事：这个教室什么时候空出来？也就是该教室最后一门课的结束时间。

假设当前课程开始时间是 s_j 。如果所有教室中，最早结束的那个教室的结束时间都还大于 s_j ，那么：

- 最早结束的都不行
- 其他结束更晚的更不可能行

所以一定没有兼容教室，只能新开。反过来，如果最早结束的那个教室满足：

$$\text{最早结束时间} \leq s_j$$

那么这个教室就可以复用。因此我们每次只需要关心：当前所有教室里，哪个最早空出来。具体实现利用最小堆：

情况 A： $s_j \geq t$ 。说明当前课程开始时，这间教室已经空出来了。于是可以复用它。

操作：

1. 弹出堆顶
| 因为原来的结束时间已经作废了
2. 把当前课程放入该教室
| 现在这个教室新的结束时间变成 f_j
3. 把新的结束时间 f_j 再插回堆中

所以这一步本质是：

取出最早空教室 → 复用 → 更新它的新结束时间

情况 B： $s_j < t$ 。说明当前课程开始时，连最早结束的教室都还没空。那就意味着：

- 所有教室都不兼容
- 当前课无法安排进已有教室

所以必须新开一个教室，把当前课程的结束时间 f_j 插入堆中。

根据上述的描述，我们可以很轻易的看出为什么要用最小堆结构了，因为我们每安排一个新的讲座，其实就是在反复做一个操作——找最早结束时间，本质是找最小值。对于这样的问题，如果用数组或链表，每次都得把整个数组或链表扫描一遍，十分低效，最小堆恰好满足我们每次找最小值、和每次更新值的情景，因为最小堆的堆顶永远是最小的元素。

如果我们将“每间教室的当前结束时间”放进最小堆里，那么：

- 堆顶 = 最早空出来的教室
- 查看堆顶: $O(1)$
- 弹出堆顶: $O(\log d)$
- 插入新的结束时间: $O(\log d)$

算法正确性与最优性证明：

可行性：

这一点依旧显而易见

最优性：

设贪心算法最后共用了 d 间教室。我们要证明：**任何安排方案都不可能少于 d 间教室。**

设贪心算法最终共分配了 d 间教室。显然，该算法不会把两个不兼容的讲座安排到同一间教室中，因此它得到的是一个合法安排。

考虑第 d 间教室第一次被开启的时刻。设这是由于要安排某个讲座 j ，其开始时间为 s_j 。之
所以需要新开第 d 间教室，是因为此时前面已有的 $d - 1$ 间教室都不能安排讲座 j 。这说明，在这 $d - 1$ 间教室中，各自最后安排的讲座都与 j 重叠。

由于所有讲座是按开始时间递增顺序处理的，因此这些与 j 冲突的讲座都在时间 s_j 之前或之时开始，且在 s_j 时尚未结束。所以在 $s_j + \varepsilon$ 的某个极小时刻，讲座 j 与这 $d - 1$ 个讲座同时进行，从而共有 d 个讲座在该时刻重叠。

因此，该区间集合的 $depth$ 至少为 d 。而任意可行安排所需教室数都不小于区间深度，所以任何安排方案至少需要 d 间教室。

另一方面，贪心算法恰好只使用了 d 间教室，因此贪心算法达到最优值，故该算法最优。

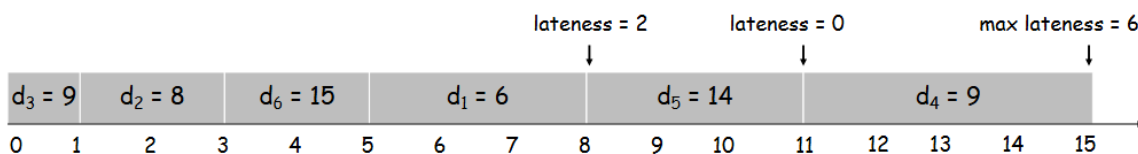
Note

其实证明思路是：证明了一定有一时刻 d 个讲座同时进行，所以教室数一定大于等于 d ，而贪心就是 d ，所以贪心最优。

最小延迟问题:

Ex:

	1	2	3	4	5	6
t_j	3	2	1	4	3	2
d_j	6	8	9	9	14	15



这是最大最小化问题，让最大的延迟尽可能的小。这道题的本质是：

所有任务都必须做完，但我们要决定先后顺序，使“最晚的那个任务”不要迟到太多。

所以它是一个**排序问题**。一旦顺序确定，并且中间不留空闲时间，那么每个任务的开始、结束时间就都被确定了。

最优安排一定没有空闲时间很好理解，不然还可以继续压缩。

最短处理时间优先不对：因为短任务虽然快，但它可能把一个截止时间特别早的长任务拖晚，从而增大最大延迟。

最小松弛量优先不对因为 $d_j - t_j$ 小并不意味着它一定应该先做，局部上的“紧迫”并不能保证全局最大延迟最小。

贪心算法思路：

按截止时间从小到大排序，依次执行任务。即：

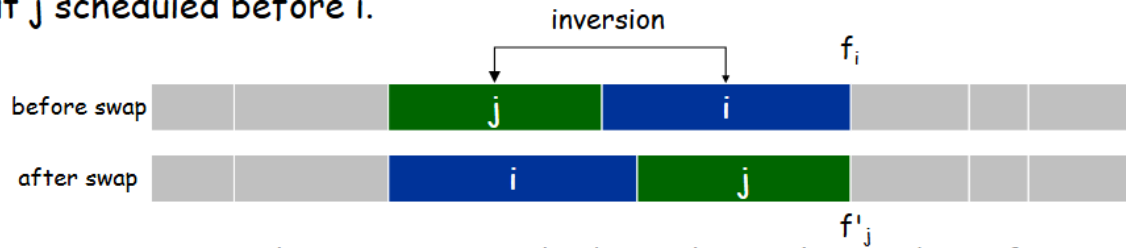
$$d_1 \leq d_2 \leq \dots \leq d_n$$

然后按这个顺序从前到后做，每个任务紧接前一个任务执行，不留空闲时间。这就是：EDF, earliest Deadline First。直觉是谁截止的时间更早，谁就更不应该被拖后腿。

贪心算法正确性证明：

关键在于找一个 *inversion* 逆序对：

but j scheduled before i .



一个任务 i 的截止时间更早，却被排在了截止时间更晚的任务 j 后面，那么这就是“排反了”的 *inversion*。

取任意一个最优调度。由“无空闲时间”观察，可设它没有空闲时间。如果这个调度中没有 *inversion*，那么它已经按 *deadline* 递增排列，即就是 EDF 调度。如果它含有 *inversion*，则一定存在一对相邻逆序任务。交换这对任务后，由上述引理知：

- 调度仍然可行；
- 最大延迟不会增大；
- 逆序对数量减少 1。

不断重复该过程。由于逆序对数有限，最终必定得到一个没有 *inversion* 的调度。这个调度就是 EDF 调度。

而在整个交换过程中，最大延迟从未增加，因此 EDF 调度的最大延迟不大于最优调度的最大延迟。由于最优调度本已最优，所以 EDF 调度也必为最优调度。

算法时间复杂度分析：

先按 *deadline* 排序：

$$O(n \log n)$$

排序后顺次执行任务并计算完成时间、延迟，只需线性扫描：

$$O(n)$$

因此总复杂度为：

$$O(n \log n)$$